A Mini Project Report Of

PCC-201-COM : Data Structures

Submitted by

Fartade Pranav Dashrath (62)

Under the Guidance of

Prof. Tejal. H. Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "**Mini Project Report**" submitted by **Fartade Pranav Dashrath (62),** on **"Implementation of Student Database Management system [Hashing]" is work done by her and submitted during 2026–2**027 academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of engineering** in computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune**.

Prof. Tejal. H. Patil
(Guide)

Dr. Lalit. V. Patil
Head of Computer Engg. Department

ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. Tejal. H. Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Dr. Lalit. V. Patil**, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

Fartade Pranav Dashrath
Student Name

ABSTRACT

The Student Database Management System is a Python-based mini project developed to demonstrate the practical implementation of hashing in Data Structures. The system is designed to manage student information, including student ID, name, department, and marks. It provides basic operations such as inserting, searching, updating, deleting, and displaying student records.

The main concept used in this project is the Hash Table, which helps determine the storage location of a student record using a hash function. When two student IDs produce the same hash index, a collision occurs. The system handles such collisions using the Linear Probing technique, which searches for the next available position in the hash table.

The project provides a simple, menu-driven interface that enables users to perform student record management operations. It demonstrates how hashing can support efficient data storage and retrieval, particularly when records are accessed using unique student IDs.

This project is developed for educational purposes and helps students understand hash functions, collision resolution, record management, and the practical application of hashing algorithms. It provides a foundation for developing more advanced student database applications in the future.

Acknowledgement i
Abstract ii
Content iii
List Of Figures iv

TABLE OF CONTENTS

SR.NO.	CONTENT	PAGE NO.
1	Introduction a) Overview b) Problem Statement c) Objective	1
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	5
5	System Design a) Existing system b) Proposed system c) ER Diagram d) System code	6
6	Result	11
7	Application	13
8	Future Scope	14
9	Conclusion	15
10	References	16

LIST OF FIGURES

Figure	Description	Page No.
Fig 4.1	UML Case Diagram	05
Fig 5.1	System Design	08
Fig 5.2	System Code	09
Fig 5.3	Result	12

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	3

INTRODUCTION

1.1 Overview

The Student Database Management System is a Python-based mini project developed to demonstrate the practical implementation of hashing in Data Structures. The system is designed to store, search, update, and delete student records efficiently using a Hash Table.

In educational institutions, student information such as student ID, name, department, and marks must be maintained and retrieved whenever required. Searching for a particular student manually can become time-consuming when the number of records increases. Therefore, an organized system is required to manage student information efficiently.

The proposed system uses a Hash Table to store student records. A hash function calculates the index at which a student record should be stored based on the student's unique ID. When two student IDs produce the same index, a collision occurs. The system resolves collisions using the Linear Probing technique, which searches for the next available position in the hash table.

1.2 Objectives

The main objectives of the Student Database Management System are:

1. To develop a simple student record management application using Python.
2. To implement a Hash Table for storing student records.
3. To design a hash function that calculates the storage index using the student ID.
4. To implement Linear Probing for resolving collisions in the Hash Table.
5. To provide a facility for adding new student records.
6. To search for student records efficiently using student IDs.
7. To update student information whenever required.
8. To delete existing student records while maintaining correct search operations.
9. To display all stored student records in an organized format.
10. To understand the practical application of hashing and collision resolution in data management.

LITERATURE SURVEY

The literature survey provides an overview of previous research and developments related to hashing, hash tables, data storage, and collision resolution techniques. It helps in understanding how data can be organized and retrieved efficiently using suitable data structures and algorithms.

Year	Paper Name	Author	Conclusion
1957	Addressing for Random-Access Storage	W. Wesley Peterson	The study examined different methods of locating records in storage systems and analyzed the searching effort required to retrieve stored information.
1978	Optimal Arrangement of Keys in a Hash Table	Ronald L. Rivest	The study investigated how keys can be arranged in hash tables using open addressing to reduce the average or worst-case number of probes required to retrieve records.
1998	Linear Probing and Graphs	Donald E. Knuth	The study presented a mathematical analysis of hashing with linear probing and examined the cost of successful search operations.
2009	Linear Probing with Constant Independence	Anna Pagh, Rasmus Pagh, and Milan Ružić	The researchers showed that five-wise independent hashing is sufficient to provide constant expected-time operations under the assumptions considered in their analysis.
2015	Comparison of Hash Table Performance with Open Addressing and Closed Addressing: An Empirical Study	Dapeng Liu and Shaochun Xu	The study experimentally compared different collision resolution techniques and found that closed addressing performed better than open addressing in their tested conditions. It highlights the importance of evaluating collision resolution methods.

UML DIAGRAM

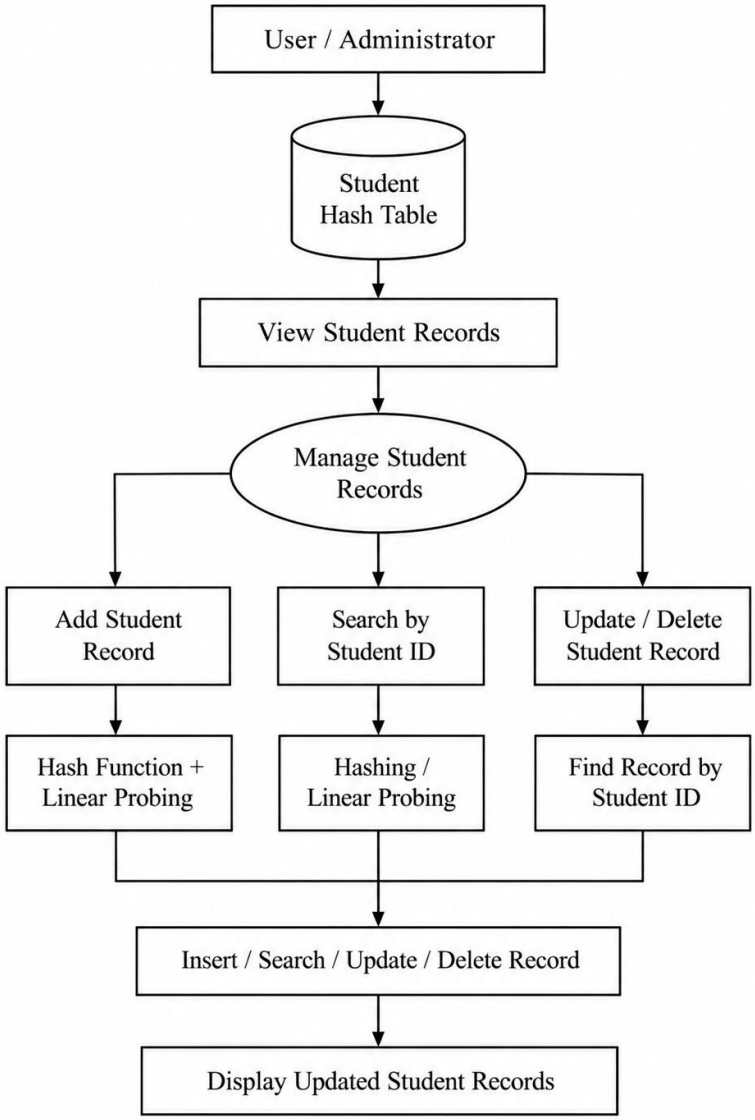


Fig. 3.1: Use Case Diagram of Student Database Management System

SYSTEM REQUIREMENT

4.1 Hardware Requirements

The following hardware configuration is recommended for running the Student Database Management System:

1. **Operating System:** Windows 10 / Windows 11 or equivalent.
2. **Processor Type:** Intel Core i3 or equivalent processor.
3. **Processor Speed:** 2.0 GHz or higher recommended.
4. **RAM:** Minimum 2 GB; 4 GB recommended.
5. **Hard Disk Space:** At least 100 MB of free storage.
6. **Input Devices:** Keyboard and mouse.
7. **Output Device:** Monitor or display screen.

4.2 Software Requirements

The following software is required to develop and execute the project:

1. **Programming Language:** Python 3.x.
2. **Code Editor / IDE:** IDLE, Visual Studio Code, PyCharm, or Jupyter Notebook.
3. **Operating System:** Windows, Linux, or macOS.
4. **Python Interpreter:** Required to execute the Python program.
5. **Data Structure:** Hash Table implemented using Python lists.
6. **Collision Resolution Technique:** Linear Probing.
7. **Data Storage:** In-memory storage for student records during program execution.

SYSTEM DESIGN

Existing System

In a traditional student record management system, student information is maintained in registers, spreadsheets, or basic computerized records. These records contain details such as student ID, name, department, and marks. Finding a particular student's information may require checking multiple records manually.

The major limitations of the existing system are:

1. **Time-Consuming Searching:** Finding a particular student may require checking many records one by one.
2. **Difficulty in Record Management:** Adding, updating, and deleting student records can require additional effort.
3. **Data Organization Problems:** Records may not be arranged in a suitable structure for efficient retrieval.
4. **Duplicate Records:** Without proper validation, the same student may be entered more than once.

Proposed System

The proposed Student Database Management System is a Python-based application designed to store, search, update, delete, and display student records using a Hash Table.

Each student record contains a unique Student ID, student name, and marks. A hash function calculates the initial storage index using the Student ID. If the calculated index is already occupied by another student record, the system uses Linear Probing to find the next available position in the Hash Table.

1. **Student Record Insertion:** Adds new student records containing Student ID, name, and marks.
2. **Hash Function:** Calculates the initial index for storing student records in the Hash Table.
3. **Collision Resolution:** Applies Linear Probing when two student IDs produce the same hash index.

Entity Relationship Diagram:

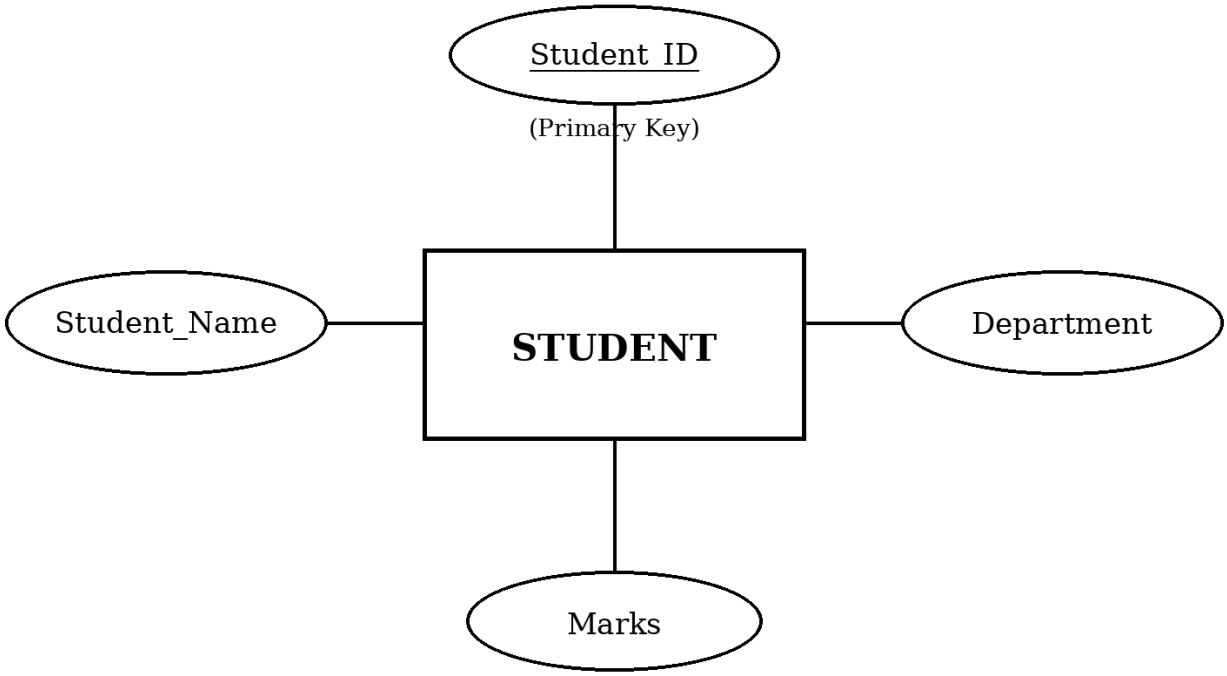
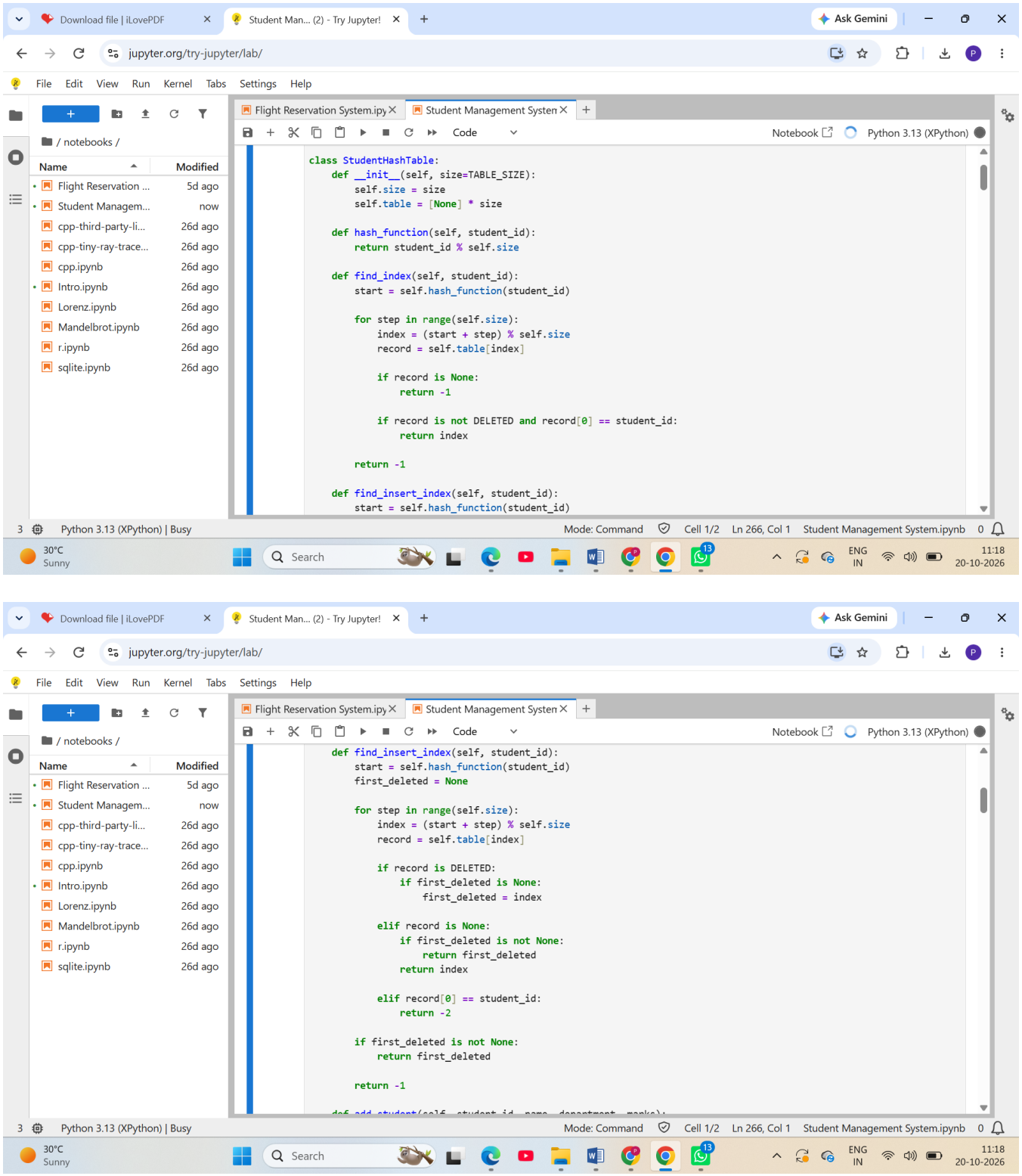


Fig. 5.1: ER Diagram of Student Database Management System

Student_ID is used as the key for hashing; collisions are handled using Linear Probing.

SYSTEM CODE



```
class StudentHashTable:
    def __init__(self, size=TABLE_SIZE):
        self.size = size
        self.table = [None] * size

    def hash_function(self, student_id):
        return student_id % self.size

    def find_index(self, student_id):
        start = self.hash_function(student_id)

        for step in range(self.size):
            index = (start + step) % self.size
            record = self.table[index]

            if record is None:
                return -1

            if record is not DELETED and record[0] == student_id:
                return index

        return -1

    def find_insert_index(self, student_id):
        start = self.hash_function(student_id)
        first_deleted = None

        for step in range(self.size):
            index = (start + step) % self.size
            record = self.table[index]

            if record is DELETED:
                if first_deleted is None:
                    first_deleted = index

            elif record is None:
                if first_deleted is not None:
                    return first_deleted
                return index

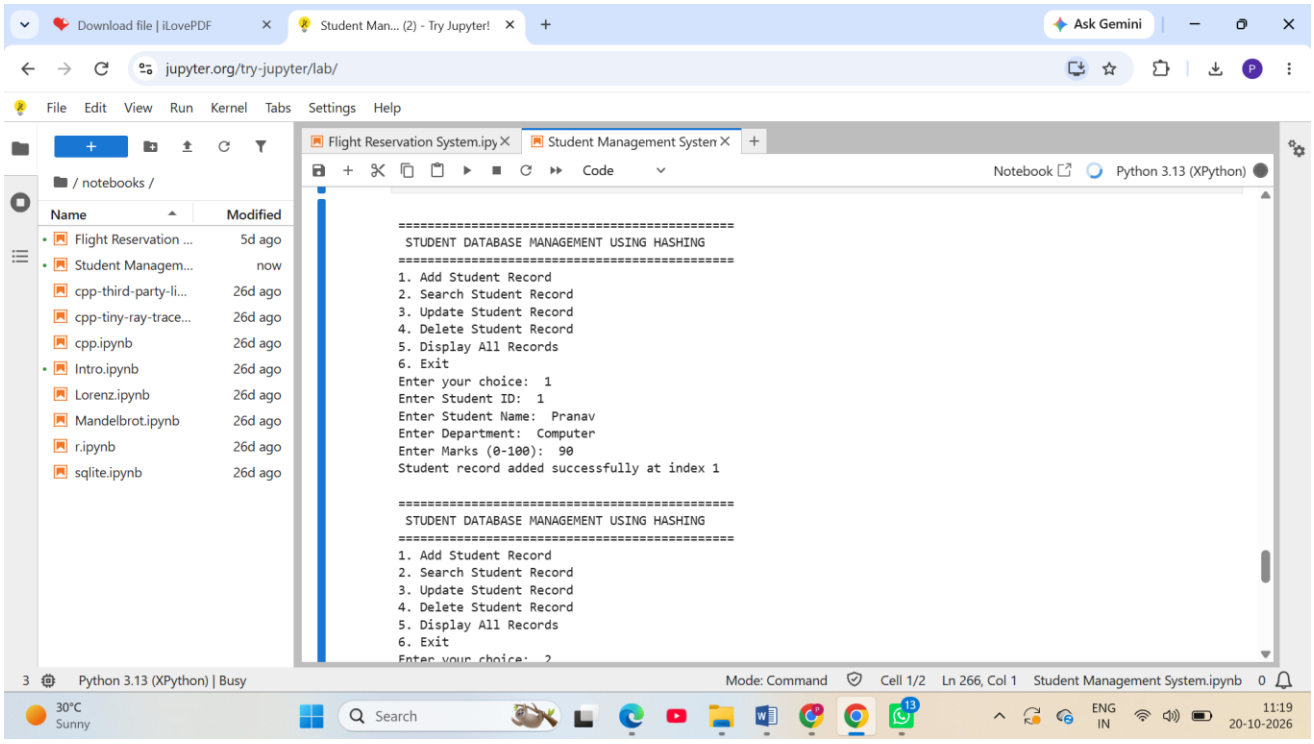
            elif record[0] == student_id:
                return -2

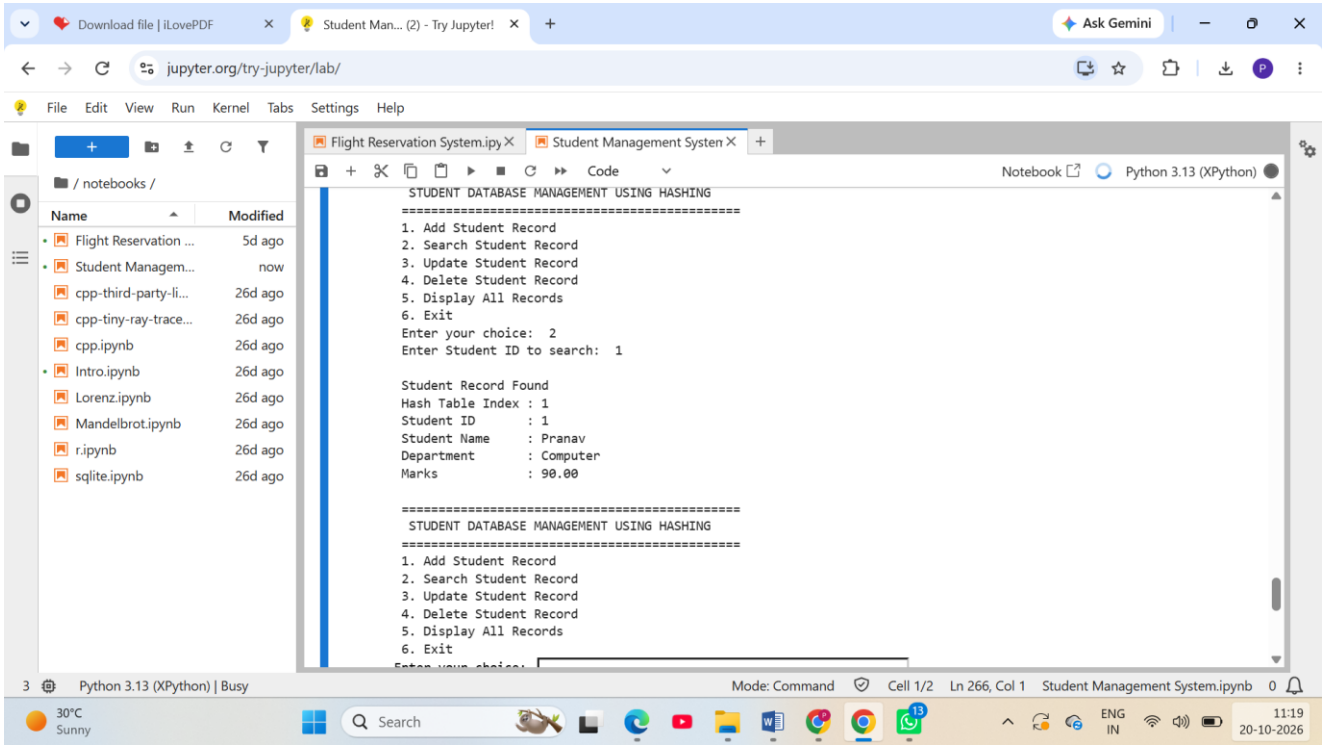
        if first_deleted is not None:
            return first_deleted

        return -1
```

Fig 5.2: System code

RESULT


Download file | iLovePDF
Student Man... (2) - Try Jupyter!
Ask Gemini
jupyter.org/try-jupyter/lab/
File Edit View Run Kernel Tabs Settings Help
Flight Reservation System.ipy
Student Management Systen
Code
Notebook
Python 3.13 (XPython)
/ notebooks /
Name
Modified
Flight Reservation ... 5d ago
Student Managem... now
cpp-third-party-li... 26d ago
cpp-tiny-ray-trace... 26d ago
cpp.ipynb 26d ago
Intro.ipynb 26d ago
Lorenz.ipynb 26d ago
Mandelbrot.ipynb 26d ago
r.ipynb 26d ago
sqlite.ipynb 26d ago
==========
STUDENT DATABASE MANAGEMENT USING HASHING
==========
1. Add Student Record
2. Search Student Record
3. Update Student Record
4. Delete Student Record
5. Display All Records
6. Exit
Enter your choice: 1
Enter Student ID: 1
Enter Student Name: Pranav
Enter Department: Computer
Enter Marks (0-100): 90
Student record added successfully at index 1
==========
STUDENT DATABASE MANAGEMENT USING HASHING
==========
1. Add Student Record
2. Search Student Record
3. Update Student Record
4. Delete Student Record
5. Display All Records
6. Exit
Python 3.13 (XPython) | Busy
Mode: Command
Cell 1/2
Ln 266, Col 1
Student Management System.ipynb
30°C Sunny
Search
ENG IN
11:19 20-10-2026


Download file | iLovePDF
Student Man... (2) - Try Jupyter!
Ask Gemini
jupyter.org/try-jupyter/lab/
File Edit View Run Kernel Tabs Settings Help
Flight Reservation System.ipy
Student Management Systen
Code
Notebook
Python 3.13 (XPython)
/ notebooks /
Name
Modified
Flight Reservation ... 5d ago
Student Managem... now
cpp-third-party-li... 26d ago
cpp-tiny-ray-trace... 26d ago
cpp.ipynb 26d ago
Intro.ipynb 26d ago
Lorenz.ipynb 26d ago
Mandelbrot.ipynb 26d ago
r.ipynb 26d ago
sqlite.ipynb 26d ago
STUDENT DATABASE MANAGEMENT USING HASHING
==========
1. Add Student Record
2. Search Student Record
3. Update Student Record
4. Delete Student Record
5. Display All Records
6. Exit
Enter your choice: 2
Enter Student ID to search: 1
Student Record Found
Hash Table Index : 1
Student ID : 1
Student Name : Pranav
Department : Computer
Marks : 90.00
==========
STUDENT DATABASE MANAGEMENT USING HASHING
==========
1. Add Student Record
2. Search Student Record
3. Update Student Record
4. Delete Student Record
5. Display All Records
6. Exit
Python 3.13 (XPython) | Busy
Mode: Command
Cell 1/2
Ln 266, Col 1
Student Management System.ipynb
30°C Sunny
Search
ENG IN
11:19 20-10-2026

APPLICATIONS

The Student Database Management System demonstrates the practical application of hashing in student record management. It provides a simple method for storing, searching, updating, and deleting student information using a Hash Table. The concepts implemented in this project can be applied in various educational and data management systems.

One of the major applications of this system is in educational institutions. Schools, colleges, and universities maintain student information such as student ID, name, department, and marks. A Hash Table can help organize these records and retrieve student information efficiently using a unique Student ID.

Another important application is student record searching and retrieval. When a teacher or administrator needs to find a particular student's details, the system uses a hash function to calculate the initial storage index. This reduces the need to check every record sequentially when the Hash Table is appropriately organized and collisions are limited.

The system can also be used for academic record management. Student names, departments, and marks may need to be updated when corrections are required. The proposed application demonstrates how an existing student record can be located through its Student ID and modified without creating a duplicate record.

FUTURE WORK

The Student Database Management System can be further improved by adding advanced features to make it more efficient, reliable, and user-friendly. In the future, the system can be connected to a database such as MySQL or SQLite to store student records permanently. This would allow student information to remain available even after the program is closed or restarted.

A graphical user interface (GUI) can be developed using Python libraries such as Tkinter or PyQt to make the application easier to use. Instead of interacting through a menu-driven console, users could add, search, update, delete, and view student records using buttons, forms, and tables.

The system can also be enhanced by introducing additional student information, such as attendance, examination results, contact details, and academic performance. New features could include attendance tracking, marksheet generation, department-wise student lists, and automatic report generation. These features would make the application more useful for educational institutions.

CONCLUSION

The Student Database Management System using Hashing was developed successfully using the Python programming language. The main objective of this project was to demonstrate the practical application of Hash Tables and collision resolution techniques in managing student records. The system allows users to add, search, update, delete, and display student information using a unique Student ID.

The project implements a hash function to calculate the storage index for each student record. When two different Student IDs generate the same hash index, the Linear Probing technique is used to find the next available position in the Hash Table. This prevents existing records from being overwritten and demonstrates how collisions can be handled during data insertion. The system also supports searching, updating, and deleting records while maintaining correct search operations.

The developed application provides a simple, menu-driven interface that makes student record management easy to understand and operate. Through this project, the importance of hashing in data storage and retrieval has been understood. It also provides practical knowledge of Python programming, lists, hash functions, collision resolution, and basic record management operations.

REFERENCE

[1] W. W. Peterson, "Addressing for Random-Access Storage," *IBM Journal of Research and Development*, vol. 1, no. 2, pp. 130–146, 1957. DOI: https://doi.org/10.1147/RD.12.0130

[2] R. L. Rivest, "Optimal Arrangement of Keys in a Hash Table," *Journal of the ACM*, vol. 25, no. 2, pp. 200–209, 1978. DOI: https://doi.org/10.1145/322063.322065

[3] D. E. Knuth, "Linear Probing and Graphs," *Algorithmica*, vol. 22, no. 4, pp. 561–568, 1998. DOI: https://doi.org/10.1007/PL00009240

[4] A. Pagh, R. Pagh, and M. Ružić, "Linear Probing with Constant Independence," *SIAM Journal on Computing*, vol. 39, no. 3, pp. 1107–1120, 2009. DOI: https://doi.org/10.1137/070702278

[5] D. Liu and S. Xu, "Comparison of Hash Table Performance with Open Addressing and Closed Addressing: An Empirical Study," *International Journal of Networked and Distributed Computing*, vol. 3, no. 1, pp. 60–68, 2015. DOI: https://doi.org/10.2991/ijndc.2015.3.1.7